

Project: Secure AWS Environment with IAM Governance — Full Step-by-Step

Prerequisites

- An AWS account and a user with **AdministratorAccess** to perform initial setup.
- AWS CLI configured locally (`aws configure`) (optional but helpful).

Overview of what we will create

1. IAM Groups: Developers, Auditors, Uploaders, Finance
2. Custom policies (JSON):
 - `DevProjectPolicy` — least privilege EC2 (only on tagged resources) + S3 limited (no delete)
 - `AuditorReadOnly` — read-only (or attach AWS managed `ReadOnlyAccess`)
 - `UploaderPolicy` — upload-only to a specific S3 prefix/bucket
 - `FinanceBillingPolicy` — billing-only view
3. IAM Users and assign to groups
4. MFA & Password Policy
5. IAM Role for EC2 (`EC2ProjectRole`) with S3 access (attach to EC2 instance)
6. Tests: allowed/denied actions, assume-role
7. Cleanup steps

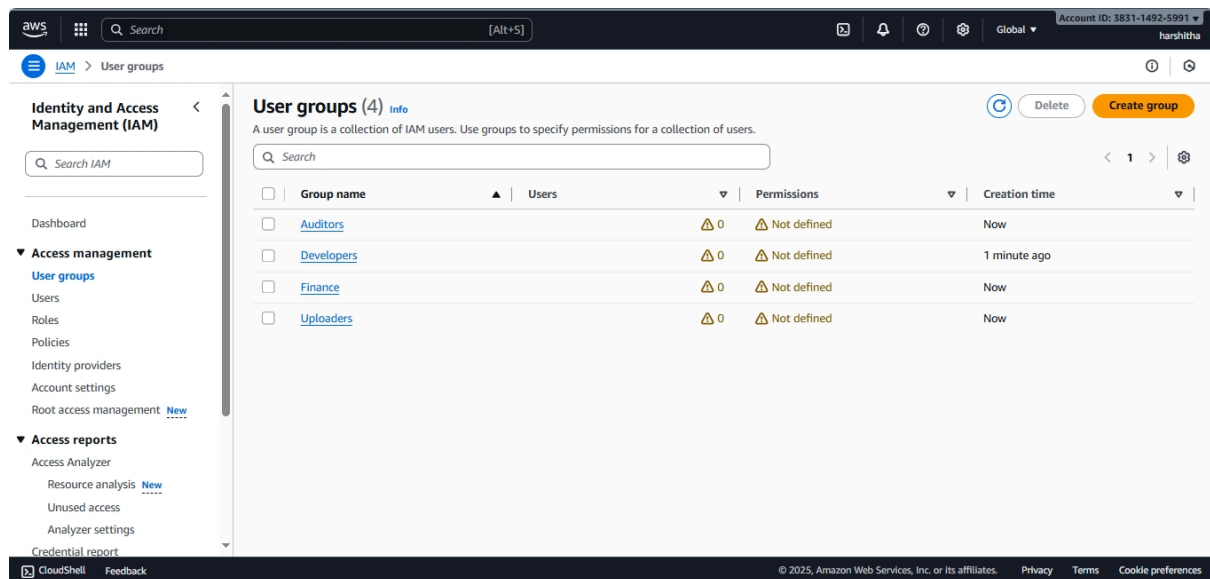
STEP 1 — Create IAM Groups (Console / CLI)

Console

1. Go to **IAM** → **Groups** → **Create group**
2. Group name: Developers. (Repeat for Auditors, Uploaders, Finance.)
3. Don't attach managed policies yet — we'll attach custom policies.

CLI (example)

```
aws iam create-group --group-name Developers
aws iam create-group --group-name Auditors
aws iam create-group --group-name Uploaders
aws iam create-group --group-name Finance
```



STEP 2 — Create Custom Policies (JSON)

Create separate JSON files, then create policies in IAM console or via CLI.

2.1 Developer policy — DevProjectPolicy.json

- Goal: Developers can manage EC2 instances **only if those instances carry a tag Project=Projectx**(enforces resource-tagging), and can read/write S3 objects under a project prefix but **NOT** delete.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EC2ActionsOnTaggedResources",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:DescribeVolumes",
        "ec2:StartInstances",
        "ec2:StopInstances",
        "ec2:RebootInstances",
        "ec2:AttachVolume",
        "ec2:DetachVolume"
      ],
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "ec2:ResourceTag/Project": "Projectx"
        }
      }
    },
    {
      "Sid": "S3ReadWriteNoDelete",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::harshitha-projectx-prod",
        "arn:aws:s3:::harshitha-projectx-prod/*"
      ]
    }
  ]
}
```

```

    },
    {
      "Sid": "DenyS3Delete",
      "Effect": "Deny",
      "Action": [
        "s3:DeleteObject",
        "s3:DeleteObjectVersion"
      ],
      "Resource": [
        "arn:aws:s3:::harshitha-projectx-prod/*"
      ]
    }
  ]
}

```

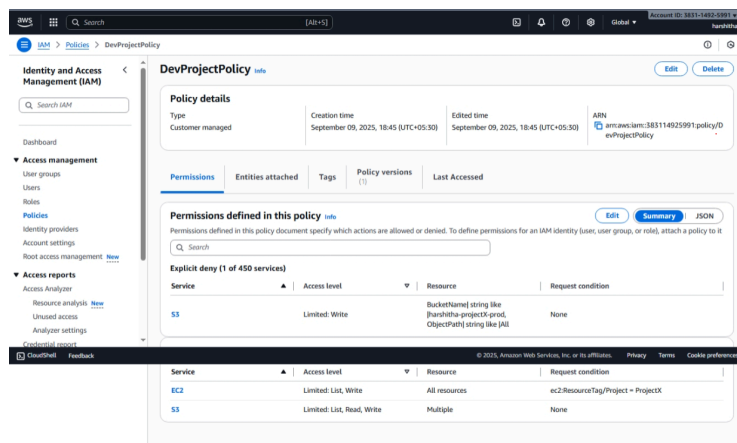
Notes:

- EC2 resource ARNs are complex; to enforce tag-based control we allow the actions but require the instance to have Project=Projectx tag.
- Developer policy denies deletes on S3 for added safety.

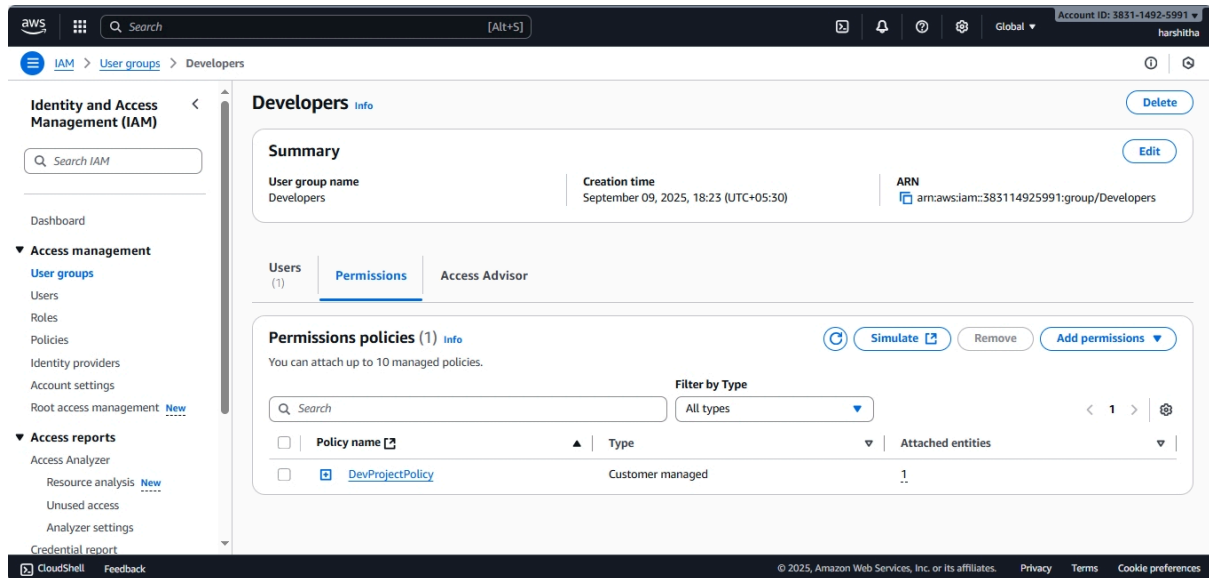
Create it in console → Policies → Create policy → JSON → paste → Review policy DevProjectPolicy.

CLI:

aws iam create-policy --policy-name DevProjectPolicy --policy-document file:///DevProjectPolicy.json



Attach this policy to the Developers group

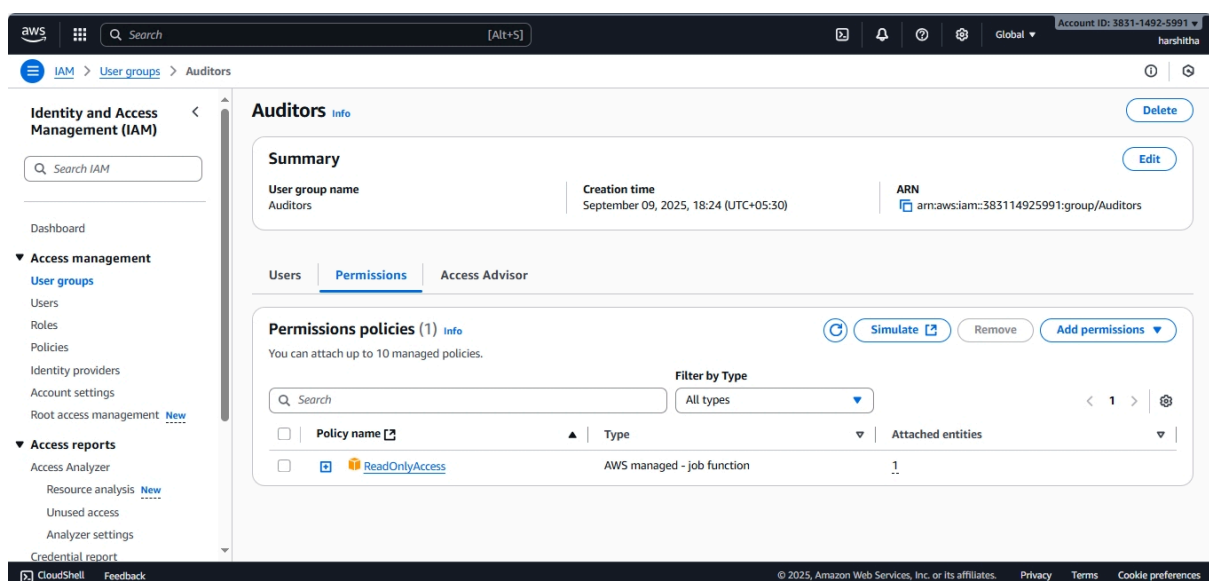


2.2 Auditor — use AWS managed ReadOnlyAccess (recommended)

Console: attach managed policy ReadOnlyAccess to Auditors group.

CLI:

```
aws iam attach-group-policy --group-name Auditors --policy-arn  
arn:aws:iam::aws:policy/ReadOnlyAccess
```

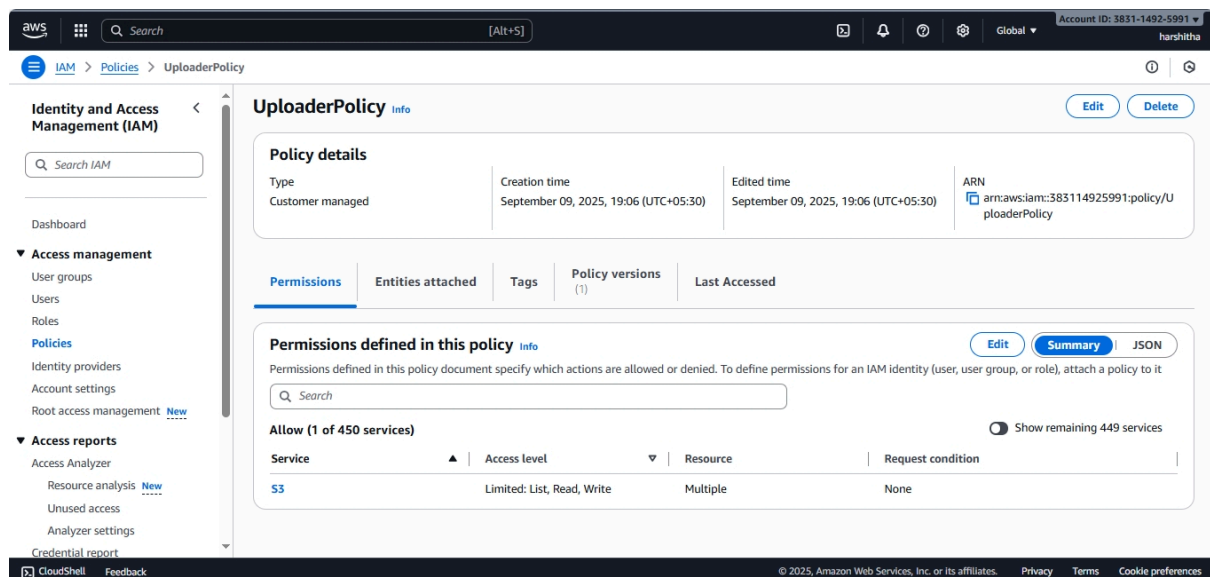


2.3 Uploader policy — UploaderPolicy.json

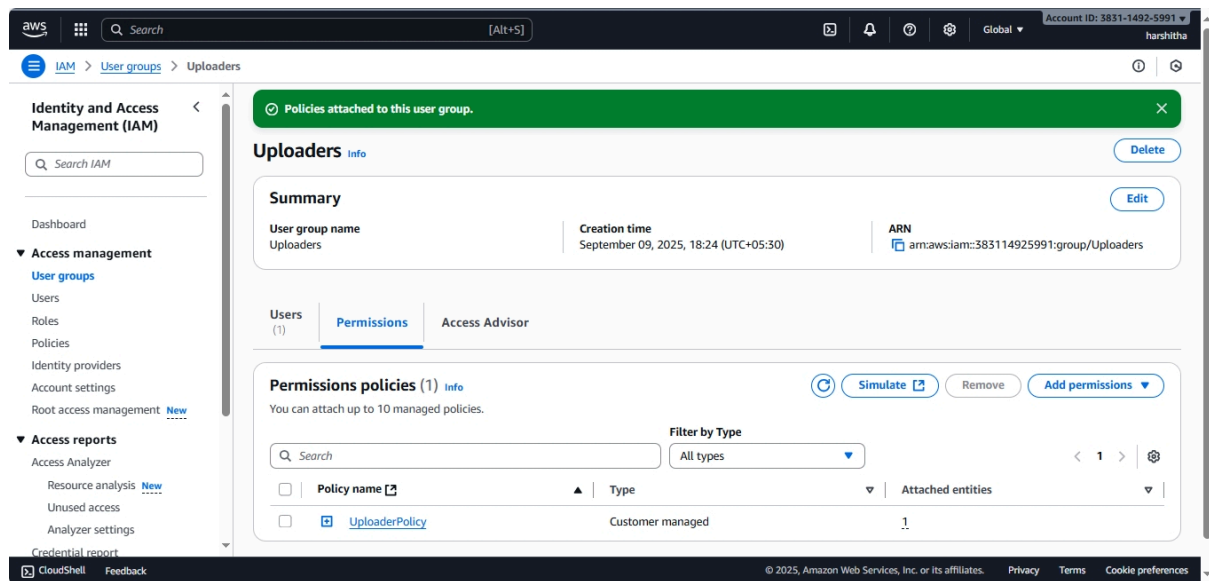
Allow PUT/GET only to a specific bucket prefix (backups/).

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowListBucket",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": ["arn:aws:s3:::harshitha-projectx-backup"]
    },
    {
      "Sid": "AllowPutGetObjects",
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject"],
      "Resource": ["arn:aws:s3:::harshitha-projectx-backup/backups/*"]
    }
  ]
}
```

Create policy (console or CLI).



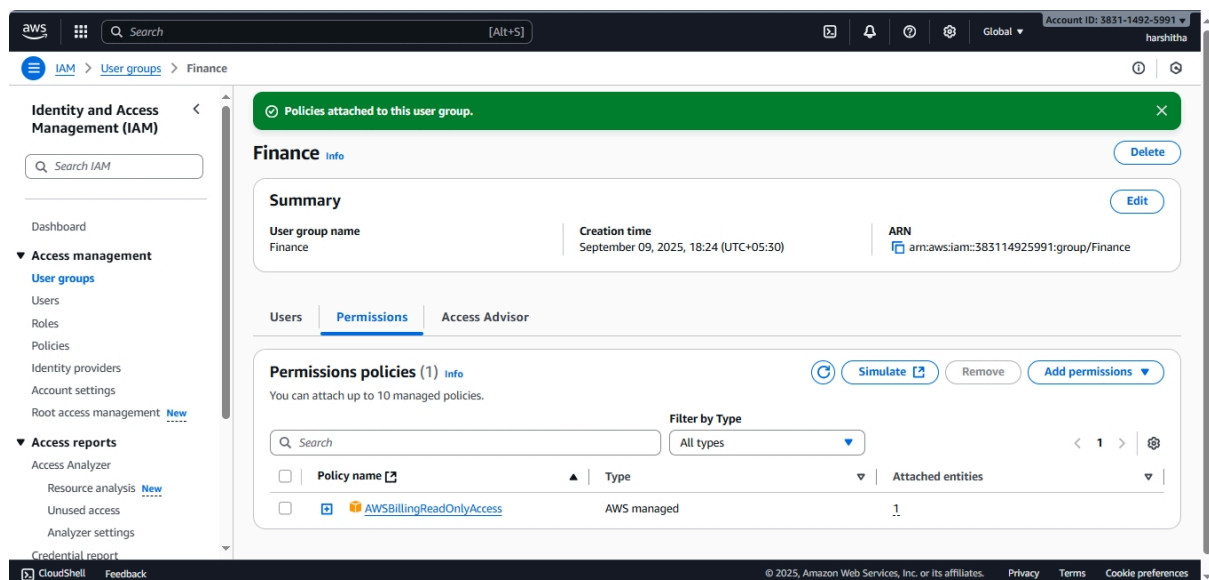
Attach this policy to the Uploaders group



2.4 Finance billing policy

Use managed Billing related policy or attach `AWSBillingReadOnlyAccess`. Note that to view billing console you need account root or permissions; attaching `aws-portal` permissions is sensitive — simplest approach: add user to group with AWS managed policy `Billing`.

Console: attach `AWSBillingReadOnlyAccess` to Finance group (or create custom policy that allows `aws-portal:*Billing`).



STEP 3 — Create IAM Users and Assign to Groups

Console

1. IAM → Users → Add user
2. Create dev-user1 with **Console access** and **Programmatic access** as needed.
3. Add to group Developers.
4. Repeat for auditor-user1 → group Auditors,
uploader-user1 → group Uploaders,
finance-user1 → group Finance.
5. For console access give them a password reset on first sign-in.

CLI example to create user and add to group:

```
aws iam create-user --user-name dev-user1
```

```
aws iam create-login-profile --user-name dev-user1 --password 'TempPassword123!'  
--password-reset-required
```

```
aws iam add-user-to-group --user-name dev-user1 --group-name Developers
```

(Use secure password; in production use proper user onboarding.)

After creating programmatic access keys, store them safely (displayed once only).

aws

Search

[Alt+S]

Global

Account ID: 3831-1492-5991

harshitha

IAM > User groups > Finance

Identity and Access Management (IAM)

Search IAM

Dashboard

Access management

User groups

Users

Roles

Policies

Identity providers

Account settings

Root access management

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Credential report

Finance

Summary

User group name: Finance

Creation time: September 09, 2025, 18:24 (UTC+05:30)

ARN: arn:aws:iam:383114925991:group/Finance

Users (1)

Permissions

Access Advisor

Users in this group (1)

Search

1

Groups

Last activity

Creation time

finance-user1

1

None

Now

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws

Search

[Alt+S]

Global

Account ID: 3831-1492-5991

harshitha

IAM > User groups > Developers

Identity and Access Management (IAM)

Search IAM

Dashboard

Access management

User groups

Users

Roles

Policies

Identity providers

Account settings

Root access management

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Credential report

Developers

Summary

User group name: Developers

Creation time: September 09, 2025, 18:23 (UTC+05:30)

ARN: arn:aws:iam:383114925991:group/Developers

Users (1)

Permissions

Access Advisor

Users in this group (1)

Search

1

Groups

Last activity

Creation time

dev-user1

1

None

1 minute ago

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws

Search

[Alt+S]

Global

Account ID: 3831-1492-5991

harshitha

IAM > User groups > Auditors

Identity and Access Management (IAM)

Search IAM

Dashboard

Access management

User groups

Users

Roles

Policies

Identity providers

Account settings

Root access management

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Credential report

Auditors

Summary

User group name: Auditors

Creation time: September 09, 2025, 18:24 (UTC+05:30)

ARN: arn:aws:iam:383114925991:group/Auditors

Users (1)

Permissions

Access Advisor

Users in this group (1)

Search

1

Groups

Last activity

Creation time

auditor-user1

1

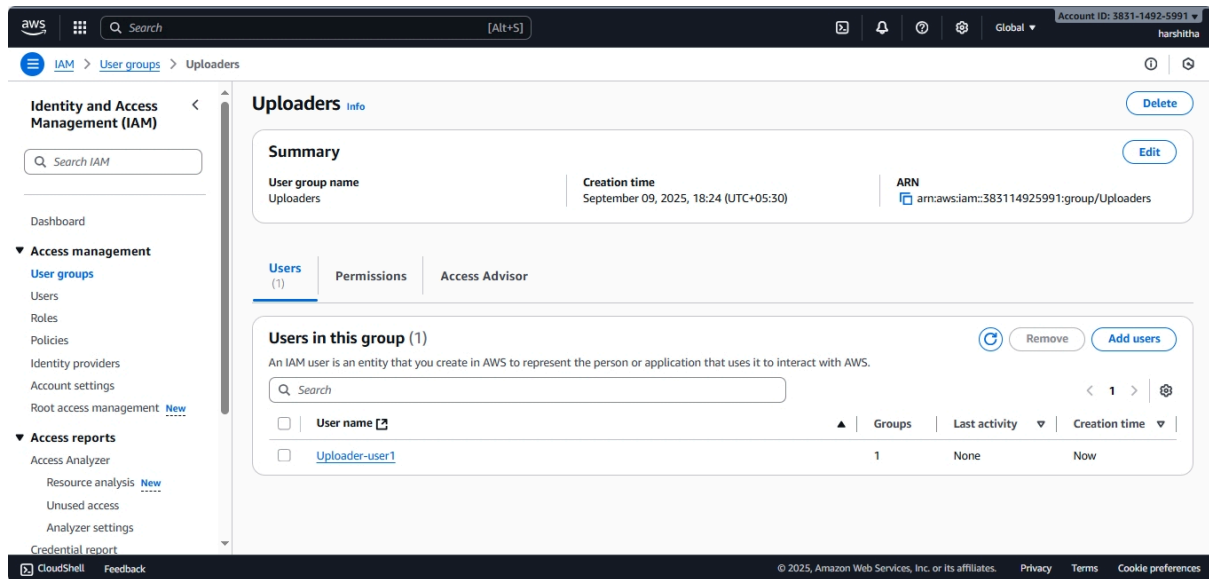
None

Now

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



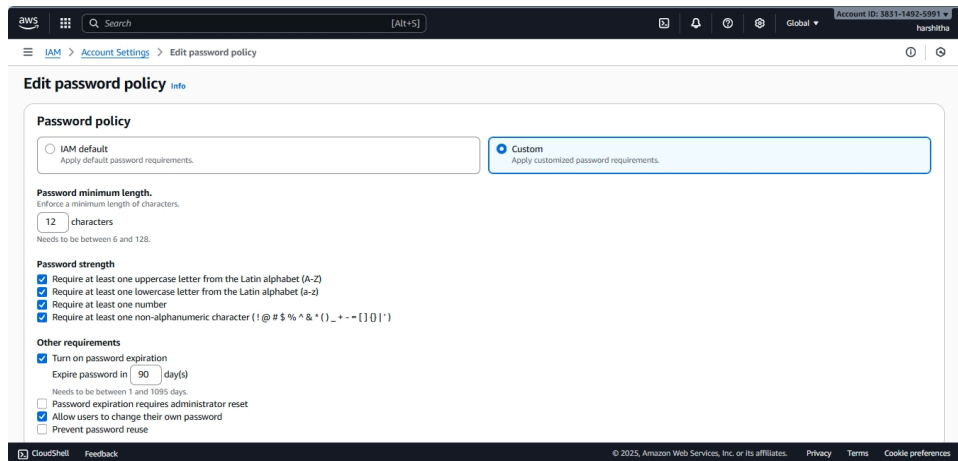
STEP 4 — Enforce MFA & Password Policy

4.1 Enforce a strong account password policy (Console)

IAM → Account settings → **Password policy** → Edit:

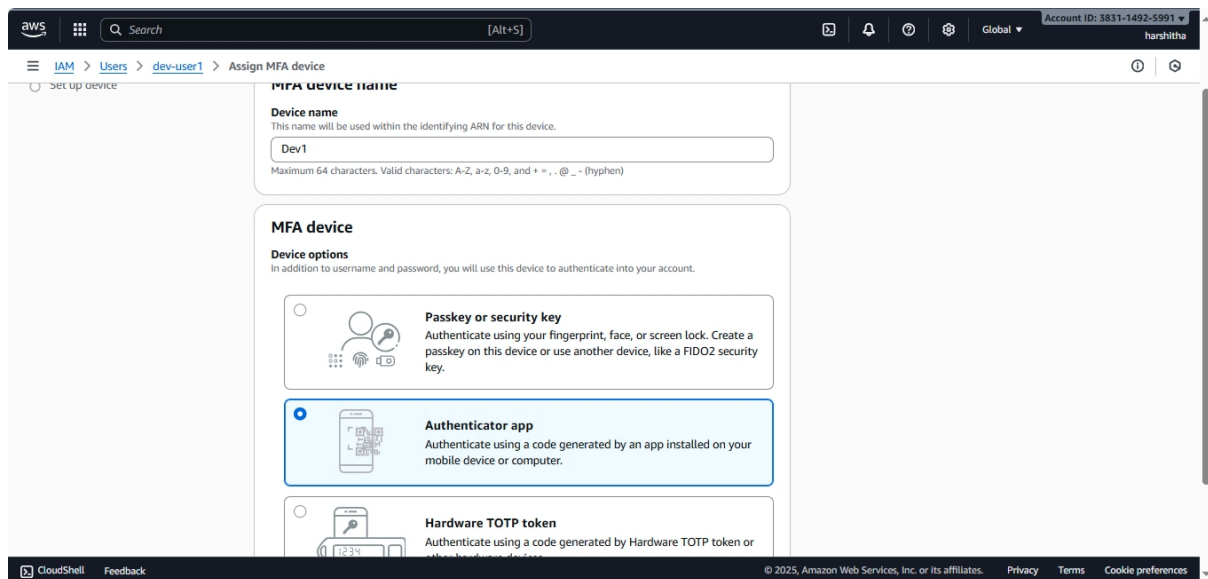
- Require at least 12 characters (or 8+), require symbols, numbers, uppercase, lowercase.
- Require password reset every 90 days (optional).
- Allow users to change password.

Save.

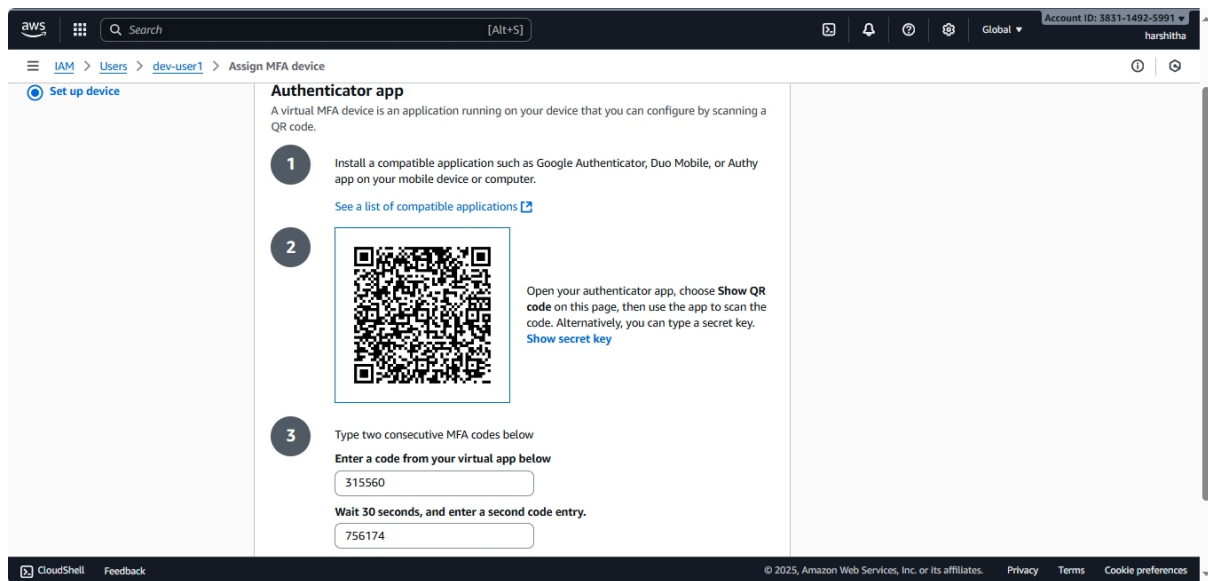


4.2 Enable MFA for each IAM user (Console)

- IAM → Users → select a user (e.g., dev-user1) → Security credentials → Manage MFA device → choose Virtual MFA → Follow steps using Google Authenticator or Authy.



- Ask the user to complete MFA setup.



Note: You can require MFA via policies for sensitive actions, but enabling it per user is good practice.

STEP 5 — Create IAM Role for EC2 (EC2ProjectRole) and instance profile

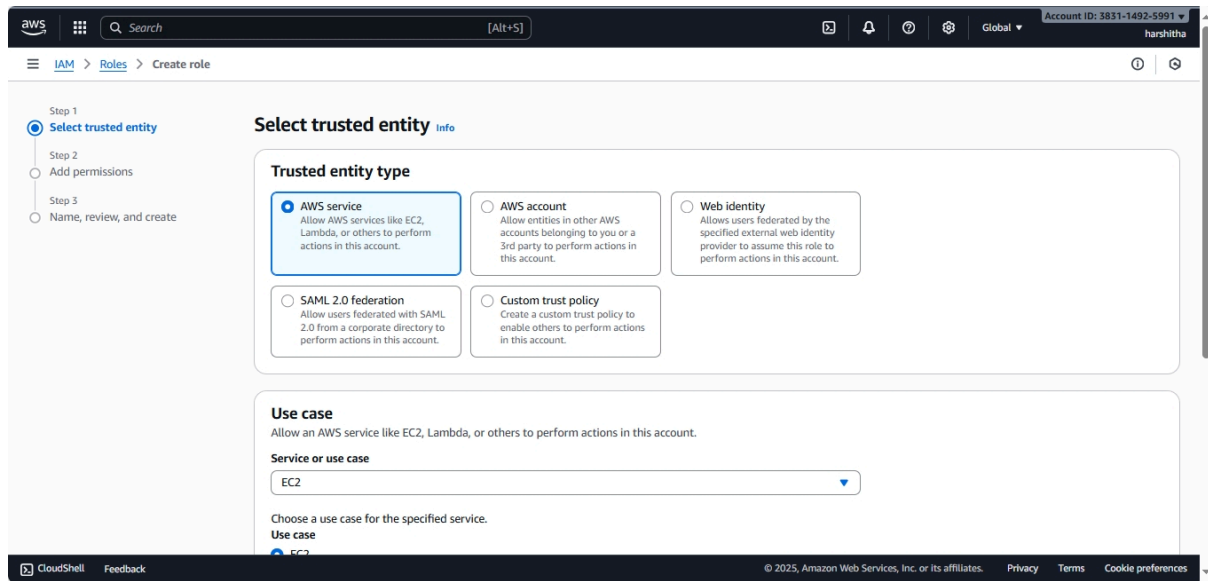
We'll create a role that EC2 instances assume to access S3 (e.g., read/write project bucket) — **no long-term keys inside instances**.

Step 1: Open IAM in Console

1. Login to AWS Management Console → Search **IAM**.
2. In left menu → Click **Roles**.
3. Click **Create role** button.

Step 2: Select trusted entity (who will use this role)

- In **Trusted entity type** → choose **AWS service**.
- In **Use case** → choose **EC2** (because EC2 instances will assume this role).



- Click **Next**.

Step 3: Attach permissions policies

- Here you decide what permissions the EC2 instance should have.

Role policy (allow S3 full to project prefix or limited actions)

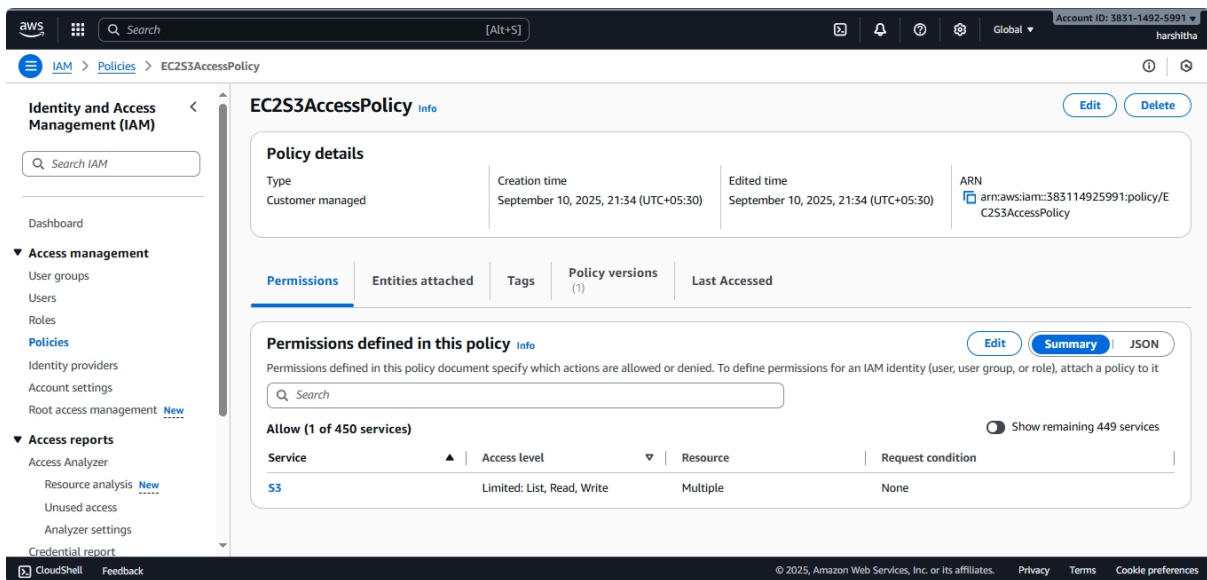
ec2-s3-access-policy.json:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::harshitha-projectx-prod",
        "arn:aws:s3:::harshitha-projectx-prod/*"
      ]
    }
  ]
}
```

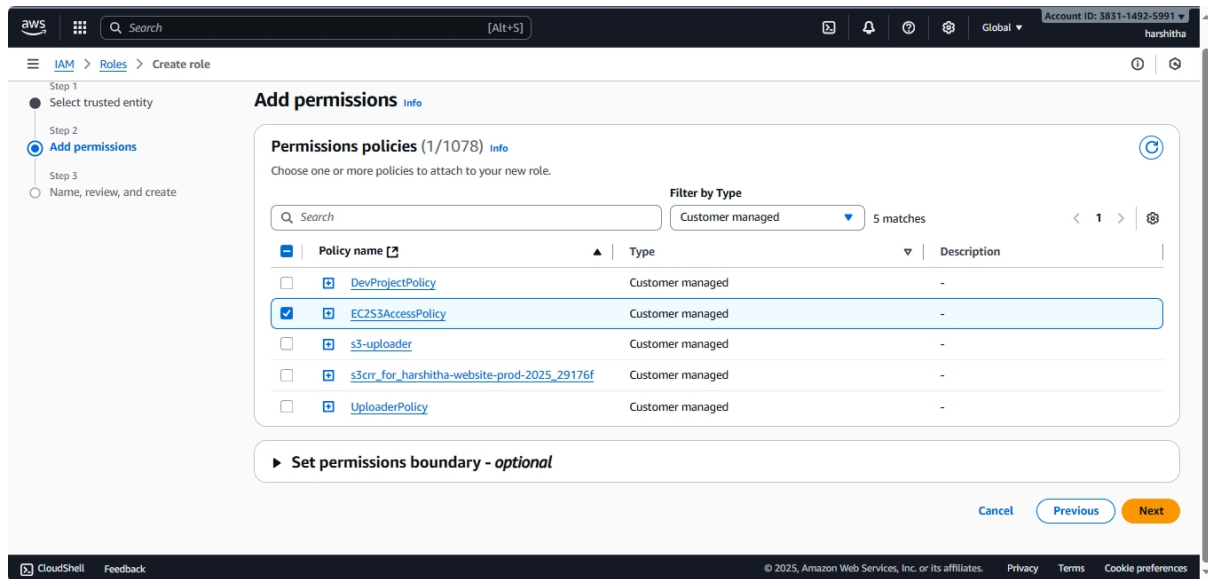
}

- Options:

- If you already have a custom policy (like your JSON), click **Create policy** (it opens in a new tab).
- In that create-policy screen:
 - Choose **JSON** tab → paste your `ec2-s3-access-policy.json`.
 - Save with name like `EC2S3AccessPolicy`.



- Come back to role creation wizard → **refresh policies** → select your new policy.



Step 4: Add tags (optional)

- You can add key-value tags (e.g., Project = ProjectX).
- Not mandatory.
- Click **Next**.

Step 5: Name and create role

- Role name → EC2ProjectRole.
- Review trust policy (should show EC2 as trusted service).
- Click **Create role**.

aws

Search

[Alt+S]

Account ID: 3831-1492-5991

harshitha

IAM > Roles > Create role

123

Step 1
Select trusted entity

Step 2
Add permissions

Step 3
Name, review, and create

Name, review, and create

Role details

Role name

Enter a meaningful name to identify this role.

EC2ProjectRole

Maximum 64 characters. Use alphanumeric and "+", "@", "-", "." characters.

Description

Add a short explanation for this role.

Allows EC2 instances to access s3

Maximum 1000 characters. Use letters [A-Z and a-z], numbers [0-9], tabs, new lines, or any of the following characters: "_", "+", "@", "-", "/", "(", ")", "#", "%", "^", ":", "~", "!", "'", "&".

Step 1: Select trusted entities

Edit

Trust policy

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": {
7         "Service": {
8           "ec2.amazonaws.com"
9         }
10      }
11    ]
12  }
13 }
14
15
16 }
```

Step 2: Add permissions

Edit

Permissions policy summary

Policy name	Type	Attached as
EC2S3AccessPolicy	Customer managed	Permissions policy

Step 3: Add tags

Add tags - optional

Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Cancel

Previous

Create role

aws

Search

[Alt+S]

Account ID: 3831-1492-5991

harshitha

IAM > Roles > EC2ProjectRole

123

Identity and Access Management (IAM)

Search IAM

Dashboard

Access management

Access reports

EC2ProjectRole

Allows EC2 instances to access s3

Delete

Edit

Summary

Creation date

September 10, 2025, 21:41 (UTC+05:30)

ARN

arn:aws:iam::383114925991:role/EC2ProjectRole

Last activity

-

Maximum session duration

1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (1)

Simulate

Remove

Add permissions

You can attach up to 10 managed policies.

Filter by Type

Search

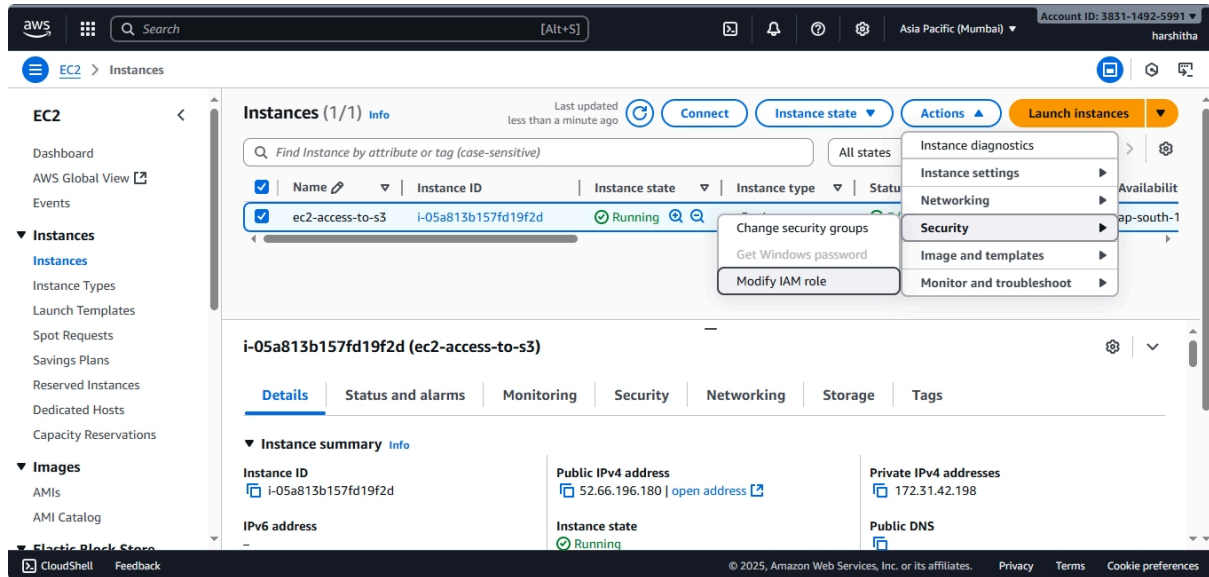
All types

Policy name	Type	Attached entities
EC2S3AccessPolicy	Customer managed	2

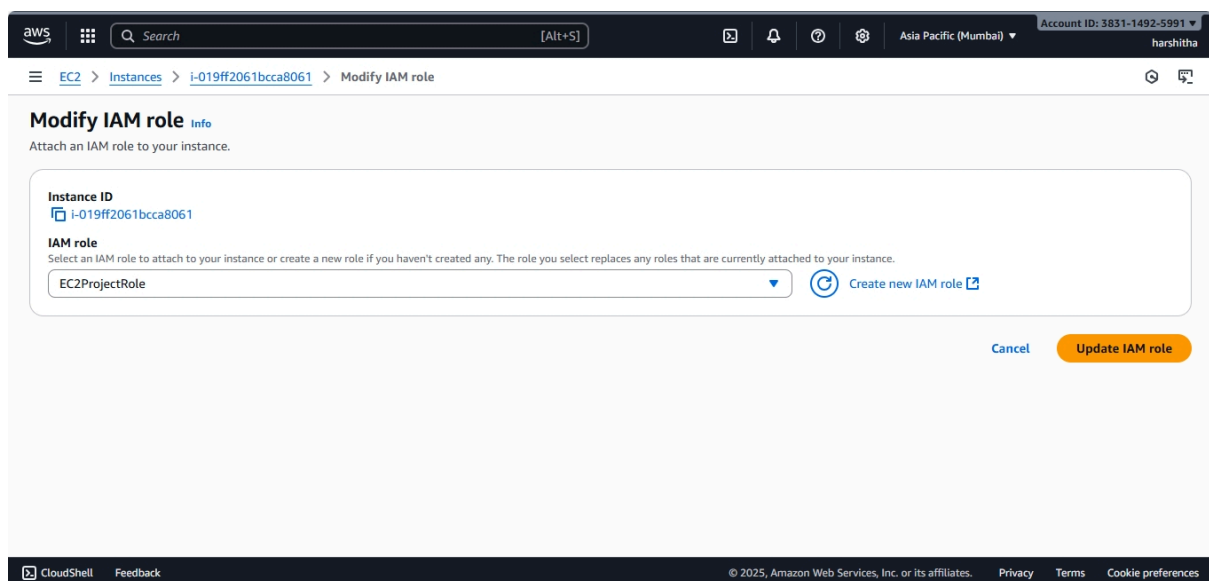
Step 6: Attach role to EC2 instance

1. For an **existing EC2 instance** →

- Go to **EC2 console** → Select your instance → **Actions** → **Security** → **Modify IAM role**.

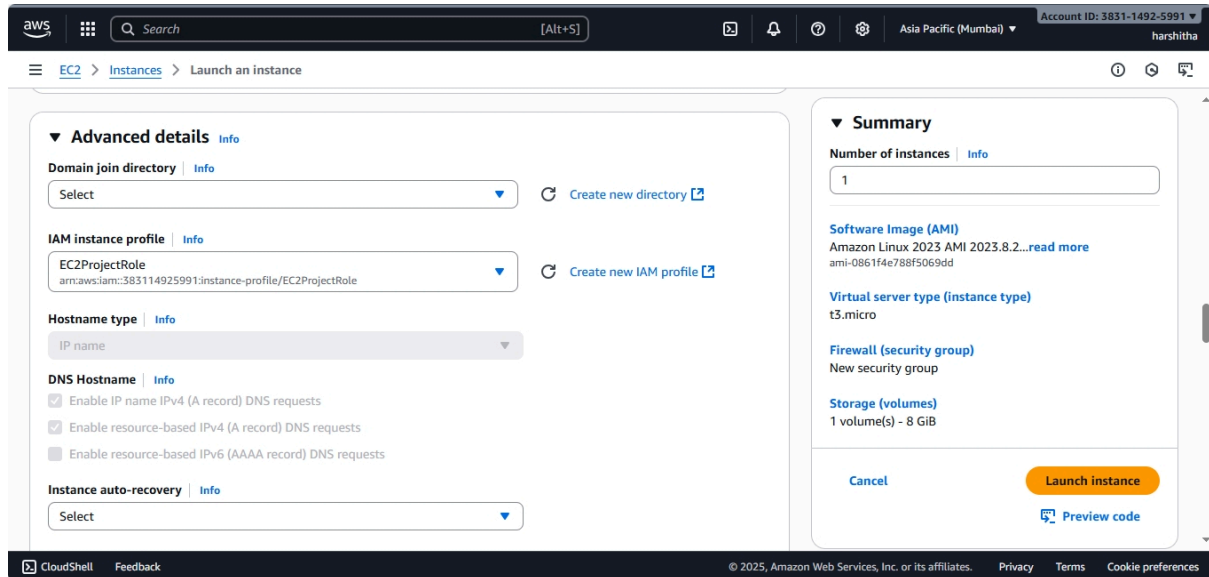


- Choose **EC2ProjectRole** → **Save**.



2. While launching a new EC2 →

- In **Step: Configure Instance details** → **IAM Role** dropdown → choose EC2ProjectRole.



Step 7: Test inside EC2

Once role is attached:

```
aws sts get-caller-identity
```

```
aws s3 ls s3://harshitha-projectx-prod --region ap-south-1(replace your bucket name)
```

(Make sure you have a bucket with objects)

The screenshot shows the AWS CloudShell interface. At the top, there's a search bar and navigation icons. The main terminal area displays the Amazon Linux 2023 logo and a URL. The user runs the command `aws sts get-caller-identity`, which returns the role ARN: `arn:aws:sts::383114925991:assumed-role/EC2ProjectRole/i-05a813b157fd19f2d`. Then, the user runs `aws s3 ls s3://harshitha-projectx-prod --region ap-south-1`, which lists two files: `devops.jpg` and `linux.png`. Below the terminal, a summary box for the role `i-05a813b157fd19f2d (ec2-access-to-s3)` shows its public and private IP addresses. The bottom of the interface includes the CloudShell logo, a feedback link, and copyright information for Amazon Web Services, Inc. (2025).

```
aws
[Alt+S]
Account ID: 3831-1492-5991
harshitha

Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Thu Sep 11 12:52:21 2025 from 13.233.177.3
[ec2-user@ip-172-31-42-198 ~]$ aws sts get-caller-identity
{
  "UserId": "AROAQVSM3N26TQK4XH2CFM:i-05a813b157fd19f2d",
  "Account": "383114925991",
  "Arn": "arn:aws:sts::383114925991:assumed-role/EC2ProjectRole/i-05a813b157fd19f2d"
}
[ec2-user@ip-172-31-42-198 ~]$ aws s3 ls s3://harshitha-projectx-prod --region ap-south-1
[ec2-user@ip-172-31-42-198 ~]$ aws s3 ls s3://harshitha-projectx-prod --region ap-south-1
2025-09-11 12:56:58      72574 devops.jpg
2025-09-11 12:56:59    246011 linux.png
[ec2-user@ip-172-31-42-198 ~]$
```

i-05a813b157fd19f2d (ec2-access-to-s3)
PublicIPs: 52.66.196.180 PrivateIPs: 172.31.42.198

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

If list works, role setup is successful.

CLI to create role and attach policy (optional)

```
aws iam create-role --role-name EC2ProjectRole --assume-role-policy-document
file://ec2-trust-policy.json
```

```
aws iam put-role-policy --role-name EC2ProjectRole --policy-name EC2S3Access
--policy-document file://ec2-s3-access-policy.json
```

Trust policy (EC2)

ec2-trust-policy.json:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Helpful CLI commands (cheat sheet)

Create a policy:

```
aws iam create-policy --policy-name DevProjectPolicy --policy-document  
file:///DevProjectPolicy.json
```

Attach policy to group:

```
aws iam attach-group-policy --group-name Developers --policy-arn  
arn:aws:iam::123456789012:policy/DevProjectPolicy
```

Create role with trust policy:

```
aws iam create-role --role-name EC2ProjectRole --assume-role-policy-document  
file:///ec2-trust-policy.json  
aws iam put-role-policy --role-name EC2ProjectRole --policy-name EC2S3Access  
--policy-document file:///ec2-s3-access-policy.json
```

STEP 8 — Tests and Validation (detailed)

Perform tests and verify:

Test Cases for Each Policy

1. Developers → DevProjectPolicy

- **Setup:**
 - Create 2 EC2 instances:
 - Instance A → Tag Project=Projectx
 - Instance B → No Project tag / different tag.
 - Create S3 bucket harshitha-projectx-prod with one test file.
- **Test Cases:**
 - Start/Stop/Describe Instance A → **should succeed**

- Start/Stop/Describe Instance B → **should fail (no Project tag)**
- Upload (PutObject) file to bucket → **should succeed**
- Download (GetObject) file → **should succeed**
- Delete file (DeleteObject) → **should fail**
- List objects in bucket → **should succeed**

CLI commands:

Describe instance

```
aws ec2 describe-instances --instance-ids <InstanceID>
```

Start instance

```
aws ec2 start-instances --instance-ids <InstanceID>
```

Upload object

```
aws s3 cp test.txt s3://harshitha-projectx-prod/
```

Try delete (should fail)

```
aws s3 rm s3://harshitha-projectx-prod/test.txt
```

2. Auditors → ReadOnlyAccess (AWS managed)

- **Setup:**
Attach AWS managed ReadOnlyAccess policy.
- **Test Cases:**
 - View EC2 instances (DescribeInstances) → **should succeed**
 - View S3 bucket list / objects → **should succeed**
 - Start/Stop/Reboot EC2 → **should fail**

- Upload/Delete S3 objects → **should fail**

CLI commands:

aws ec2 describe-instances

aws ec2 start-instances --instance-ids <InstanceID> # should fail

aws s3 ls s3://harshitha-projectx-prod # should work

aws s3 cp test.txt s3://harshitha-projectx-prod/ # should fail

3. Uploaders → UploaderPolicy

- **Setup:**
 - Bucket: harshitha-projectx-backup
 - Prefix: backups/
- **Test Cases:**
 - Upload object to backups/ → **should succeed**
 - Download object from backups/ → **should succeed**
 - Upload outside backups/ (like root or tmp/) → **should fail**
 - List bucket → **should succeed**

CLI commands:

aws s3 cp file.txt s3://harshitha-projectx-backup/backups/ # should work

aws s3 cp file.txt s3://harshitha-projectx-backup/tmp/ # should fail

aws s3 ls s3://harshitha-projectx-backup/backups/ # should work

4. Finance → Billing Policy

- **Permission:**
AWSBillingReadOnlyAccess.
- **Test Cases:**
 - Login console → Go to **Billing Dashboard** → should open and view bills.
 - Try to launch EC2 or create S3 bucket → **should fail**.
 - Check monthly cost reports, usage reports → **should succeed**.